ASSIGNMENT 8 — GIT BASICS TW LAB

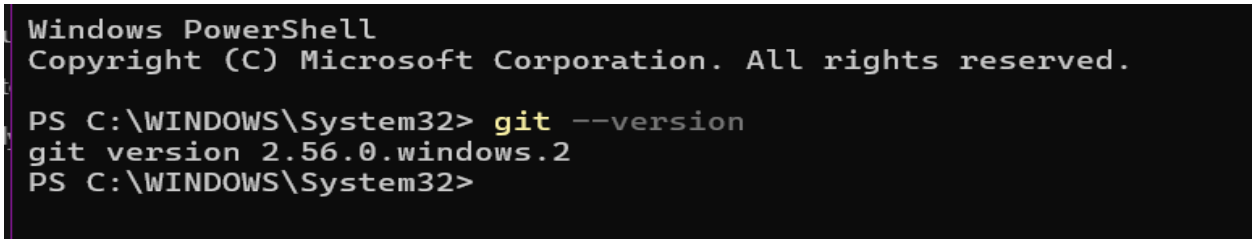
Objective

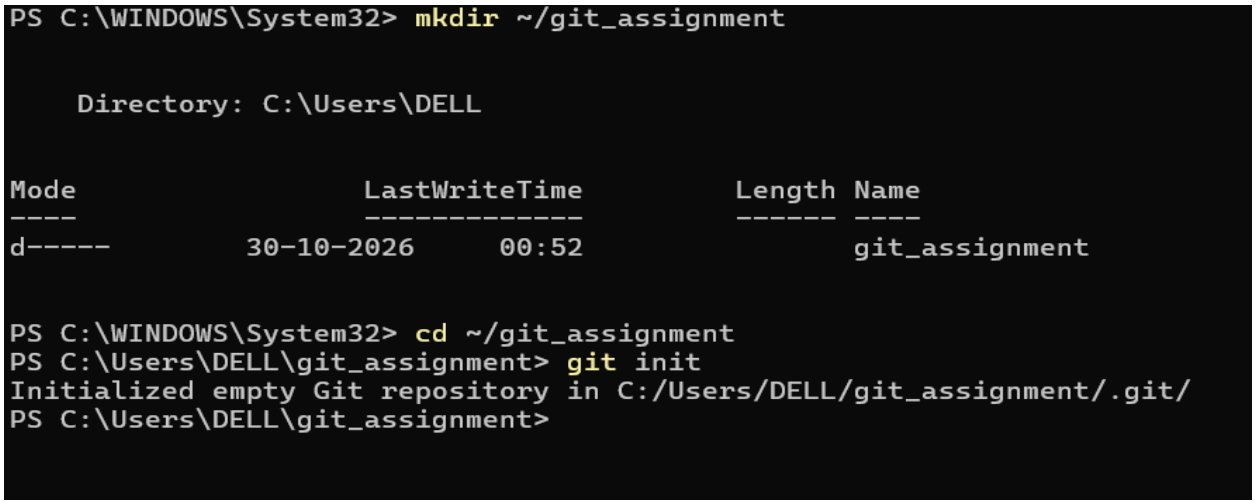
To understand and practise basic Git commands, including repository initialization, staging, committing, restoring changes, viewing history, reverting commits, and checking out earlier versions.

Task 1: Git Installation and Repository Initialization

Commands used: `git --version`, `git init`

Git was installed on Windows using Git for Windows. The installation was verified using `git --version`. A project directory named `git_assignment` was created, and `git init` was used to initialize a Git repository in that directory.





Task 2: Git Configuration

Commands used: `git config user.name`, `git config user.email`

The Git username and email address were configured locally for the repository. These settings identify the author of the commits made in the project.

```
nothing to commit, working tree clean
PS C:\Users\DELL\git_assignment> git config user.name "Gargi Kishore"
PS C:\Users\DELL\git_assignment> git config user.email "2025ucp1915@mnit.ac.in"
PS C:\Users\DELL\git_assignment> git config user.name
Gargi Kishore
PS C:\Users\DELL\git_assignment> git config user.email
2025ucp1915@mnit.ac.in
PS C:\Users\DELL\git_assignment>
```

Task 3: Creating and Staging a File

Commands used: `Set-Content`, `git add readme.txt`, `git status`

A file named `readme.txt` was created with a short description of the project. The file was added to the staging area using `git add`. The `git status` command was used to check the file's status.

```
PS C:\Users\DELL\git_assignment> Set-Content readme.txt "This project demonstrates basic Git commands."
PS C:\Users\DELL\git_assignment> git add readme.txt
PS C:\Users\DELL\git_assignment> git status
On branch master

No commits yet

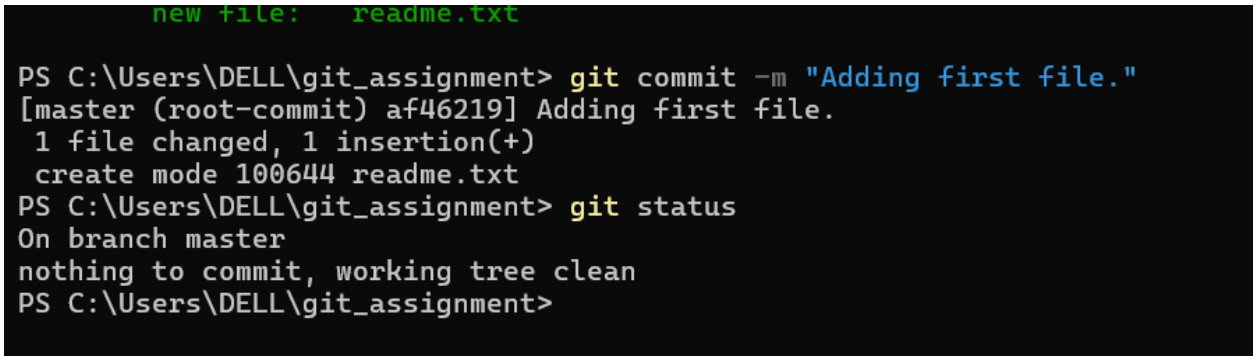
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   readme.txt

PS C:\Users\DELL\git_assignment>
```

Task 4: Creating the First Commit

Command used: `git commit -m "Adding first file."`

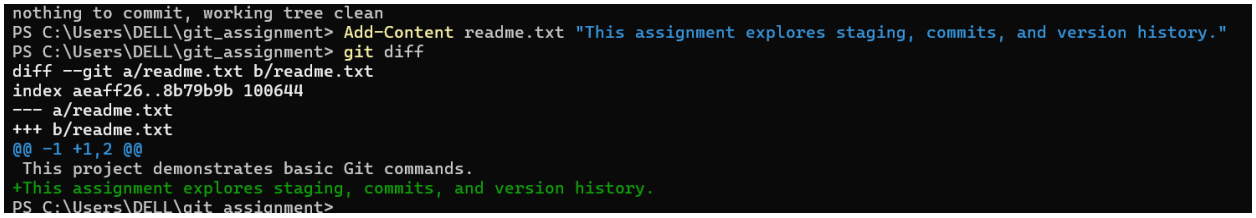
The staged `readme.txt` file was committed to the repository. A commit records a snapshot of the project's tracked files at a particular point in time.



Task 5: Modifying a File and Viewing Differences

Commands used: `Add-Content`, `git diff`

An additional line was added to `readme.txt`. The `git diff` command was used to display the changes made to the file compared with the last committed version.



Task 6: Restoring File Changes

Commands used: `git restore`, `git restore --staged`

The `git restore` command was explored to discard unstaged modifications. The `git restore --staged` command was used to remove a file from the staging area without deleting it. These commands help manage unwanted changes before committing.

```
PS C:\Users\DELL\git_assignment> git diff
diff --git a/readme.txt b/readme.txt
index aeaff26..8b79b9b 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1 +1,2 @@
 This project demonstrates basic Git commands.
+This assignment explores staging, commits, and version history.
PS C:\Users\DELL\git_assignment> git restore readme.txt
PS C:\Users\DELL\git_assignment> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\DELL\git_assignment>
```

```
PS C:\Users\DELL\git_assignment> git restore readme.txt
PS C:\Users\DELL\git_assignment> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\DELL\git_assignment> Add-Content readme.txt "Practising staged changes in Git."
PS C:\Users\DELL\git_assignment> git add readme.txt
PS C:\Users\DELL\git_assignment> git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   readme.txt

PS C:\Users\DELL\git_assignment> git restore --staged readme.txt
PS C:\Users\DELL\git_assignment> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   readme.txt

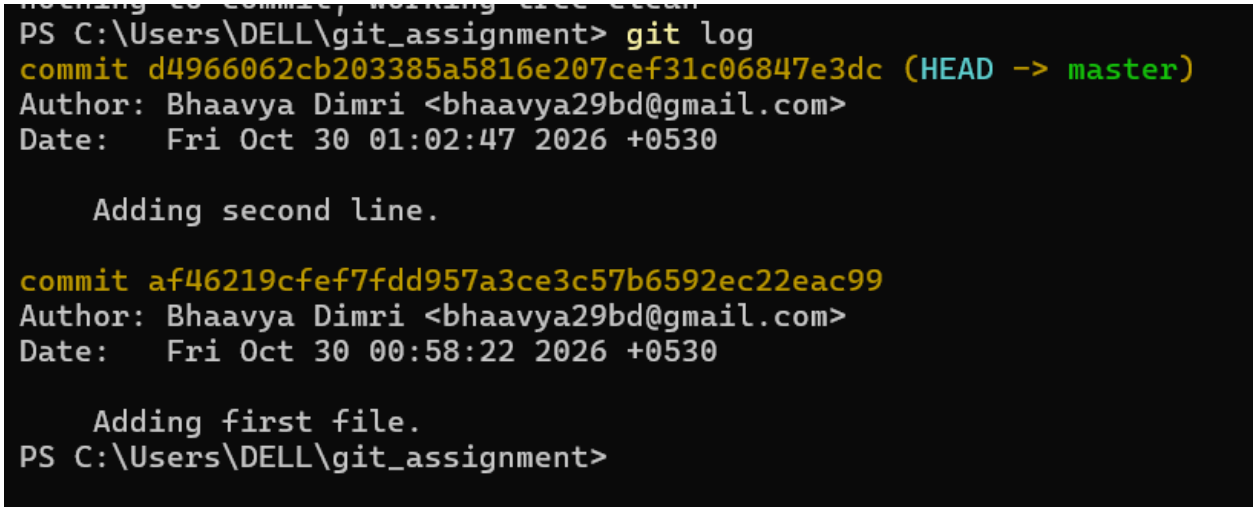
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\DELL\git_assignment>
```

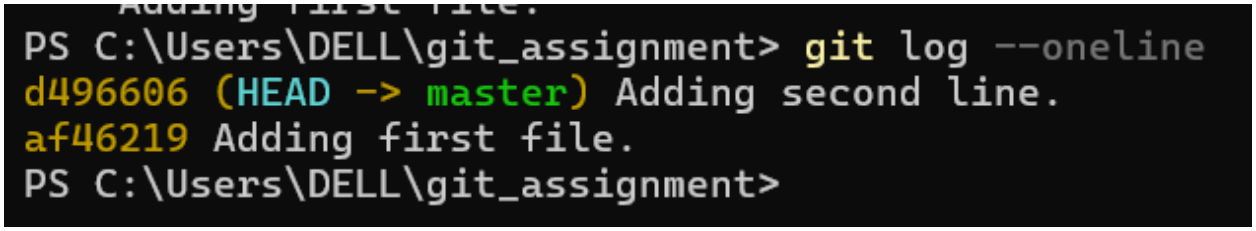
```
PS C:\Users\DELL\git_assignment> git add readme.txt
PS C:\Users\DELL\git_assignment> git commit -m "Adding second line."
[master d496606] Adding second line.
 1 file changed, 1 insertion(+)
PS C:\Users\DELL\git_assignment> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\DELL\git_assignment>
```

Task 7: Viewing Commit History

Command used: `git log --oneline`

The `git log --oneline` command was used to view the repository's commit history in a compact format. It displayed commit hashes and their corresponding messages.





Task 8: Reverting a Commit

Command used: `git revert <commit-hash>`

The `git revert` command was explored to undo the changes introduced by an earlier commit while preserving the repository's history. A conflict occurred during the revert process, and the resulting repository state was inspected and handled using Git commands.

```
PS C:\Users\DELL\git_assignment> git log --oneline
d496606 (HEAD -> master) Adding second line.
af46219 Adding first file.
PS C:\Users\DELL\git_assignment> git revert d496606
[master d6a7788] Revert "Adding second line."
 1 file changed, 1 deletion(-)
PS C:\Users\DELL\git_assignment> git log --oneline
d6a7788 (HEAD -> master) Revert "Adding second line."
d496606 Adding second line.
af46219 Adding first file.
PS C:\Users\DELL\git_assignment>
```

Task 9: Creating the Vector Dot Product Program

File created: `vecadd.py`

A Python program named `vecadd.py` was created. It accepts two vectors of integers and calculates their dot product by multiplying corresponding elements and adding the products. It also checks whether the vector sizes match the specified number of elements.

```
PS C:\Users\DELL\git_assignment> notepad vecadd.py
PS C:\Users\DELL\git_assignment> python vecadd.py
Enter the number of elements: 3
Enter the elements of the first vector:
1 2 3
Enter the elements of the second vector:
4 5 6
Dot product = 32
PS C:\Users\DELL\git_assignment> dir


    Directory: C:\Users\DELL\git_assignment


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        30-10-2026     01:12             47 readme.txt
-a----        30-10-2026     01:15            428 vecadd.py


PS C:\Users\DELL\git_assignment>
```

Task 10: Committing the Python Program

Commands used: `git add vecadd.py`, `git commit -m "Adding vector dot product program."`, `git log --oneline`

The Python program was added to the staging area and committed to the repository. The commit history was then checked to verify that the new commit was recorded.

```
PS C:\Users\DELL\git_assignment> git add vecadd.py
PS C:\Users\DELL\git_assignment> git commit -m "Adding vector dot product program."
[master 00e09d4] Adding vector dot product program.
 1 file changed, 11 insertions(+)
 create mode 100644 vecadd.py
PS C:\Users\DELL\git_assignment> git log --oneline
00e09d4 (HEAD -> master) Adding vector dot product program.
d6a7788 Revert "Adding second line."
d496606 Adding second line.
af46219 Adding first file.
PS C:\Users\DELL\git_assignment>
```

Task 11: Removing a File from the Repository

Commands used: `git rm readme.txt`, `git commit -m "Removing readme.txt."`

The `git rm` command was used to remove `readme.txt` from the working directory and stage its removal. The change was committed to record the deletion in the repository history.

```
PS C:\Users\DELL\git_assignment> git rm readme.txt
rm 'readme.txt'
PS C:\Users\DELL\git_assignment> git commit -m "Removing readme.txt."
[master 390831c] Removing readme.txt.
 1 file changed, 1 deletion(-)
 delete mode 100644 readme.txt
PS C:\Users\DELL\git_assignment> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\DELL\git_assignment>
```

Task 12: Checking Out an Earlier Commit

Command used: `git checkout <commit-hash>`

An earlier commit was checked out using its commit hash. The `dir` and `git log --oneline` commands were used to inspect the files and history at that point. Git displayed a detached HEAD message because the checkout was made directly to a commit rather than a branch.

```
00e09d4 Adding vector dot product program.
d6a7788 Revert "Adding second line."
d496606 Adding second line.
af46219 Adding first file.
PS C:\Users\DELL\git_assignment> git checkout 00e09d4
Note: switching to '00e09d4'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 00e09d4 Adding vector dot product program.
PS C:\Users\DELL\git_assignment> dir


    Directory: C:\Users\DELL\git_assignment


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        30-10-2026     01:17             47 readme.txt
-a----        30-10-2026     01:15            428 vecadd.py


PS C:\Users\DELL\git_assignment> git log --oneline
00e09d4 (HEAD) Adding vector dot product program.
d6a7788 Revert "Adding second line."
d496606 Adding second line.
af46219 Adding first file.
PS C:\Users\DELL\git_assignment>
```

Task 13: Returning to the Main Branch

Commands used: `git checkout master`, `git status`

The `git checkout master` command was used to return to the `master` branch. The `git status` command confirmed that the repository was on `master` and the working tree was clean.

```
PS C:\Users\DELL\git_assignment> git checkout master
Previous HEAD position was 00e09d4 Adding vector dot product program.
Switched to branch 'master'
PS C:\Users\DELL\git_assignment> git status
On branch master
nothing to commit, working tree clean
PS C:\Users\DELL\git_assignment>
```

Conclusion

This assignment provided practical experience with basic Git operations, including creating repositories, tracking files, staging and committing changes, inspecting differences, restoring changes, viewing commit history, reverting changes, removing files, and checking out earlier commits. It also demonstrated the use of Git to manage different versions of a project.I am using terminal on windows so commands might differ.